Angular Coding Challenge: Bhrigu

The purpose of this task is to test your knowledge on frontend development, ability to adapt to new & cutting-edge frontend technologies, learning capability, and your coding style.

Instructions

1. Create an Angular project and initialize it with Git.
2. Use spartan UI (http://spartan.ng) for styling and components.
3. Design a clean & Beautiful UI – let your creativity know no bounds. You are free to showcase CSS animations, beautiful color schemes, fonts, and styles to your heart's desire.
4. Take care of coding standards, variable naming, and readability. It is important to us.
5. Use Git for reasonable commits (make commits every 15-30 minutes).
6. All routes must be lazy loaded.
7. We are interested in seeing:
    - your skills in creating modular code and data sharing between components - across siblings (same level components) and between parent and its subcomponents.
    - Your coding standard. Clean, indented, crisp code for all code files-- HTML, CSS/ SASS, and Typescript. Add comments, as necessary. Bonus point for integrating prettier & Eslint/biomejs.
    - Good programming principles --- DRY, Object-Oriented Programming, state management concepts (use of interfaces, correct access specifiers etc.), Design patterns.
    - Use of modern Angular specific features – newer syntax, signals, signal forms, services, pipes, directives, and Observables.
8. You may use AI coding assistants during development; however, ensure that you fully understand and thoughtfully implement every line of code. **In the review meeting, we will discuss the rationale, benefits, drawbacks, and trade-offs of selected code blocks. Be prepared to explain your decisions and the reasoning behind them.**

Submission

The assignment should be completed within 24 hours of receiving it. Share the GitHub repository URL as well as a zip file of your codes (excluding node_modules). For any clarification, please feel free to contact touhid.rahman@bhrigu.tech

Good Luck!

Task

Develop a **blog and photo archive** web application. For this challenge, use the following REST API service:

https://jsonplaceholder.typicode.com

You will find a guide on how to use the API here: https://jsonplaceholder.typicode.com/guide/

Authentication

1. Create a basic signup page with First name, last name, username, password, and password confirmation.
2. The username and password should be stored in local storage when the user signs up.
3. Create a login page.
4. When the user tries to log in, the username and password should be checked against the stored username/password in the local storage.
5. If the user is logged in, their First name and last name initials should be shown on the navbar.
6. Add a logout method to log out the user, which will clean the local storage and reset the navbar.

Blog

1. Use https://jsonplaceholder.typicode.com for the REST API of this project. This website offers free REST APIs for development purposes. Read the documentation on how to use it.
2. Write individual services that will consume the /posts and /comments API in your angular app.
3. Design the home page where the first 5 posts are visible. Users can navigate to other posts using the pagination bar. Be creative!
4. When the user clicks a post, it should open the post details page. In that details page, below the article, the related comments should also be shown.
5. Users can add comments. “Add comment” page is a protected page by the logged-in guard, meaning - if the user wants to add a comment, he should be signed in. If not, he/she should be redirected to the login page. After logging in, user should be brought back to the add-comment page.
6. Once a comment is added by the user, the post details page should show that comment at the top before other comments (first comment).
7. The newly added comment should persist during the session (i.e. until the user is logging out). For this purpose, you may use a state service or local storage.

Photo Archive

1. Use the /album and /photos API to create an album viewer.
2. Show all the albums in a grid-like view.
3. Clicking on an album should show all the photos of that album in a grid.
4. The photos page should be paginated.

UI/UX & Accessibility Requirements

1. Implement responsive design with mobile-first approach using CSS Grid for the photo archive grid layout. The design must function properly on screens as small as 320px wide.

2. Implement a dark mode toggle feature with persistent user preference stored in local storage. Users should be able to switch between light and dark themes seamlessly.

3. All interactive elements must be accessible by keyboard. Users must be able to navigate and interact with the entire application using only the keyboard (Tab, Enter, Arrow keys, etc.).

4. Implement proper ARIA labels and semantic HTML. All form inputs must have associated labels, images must have alt-text, and buttons must have descriptive text.

5. Ensure WCAG 2.1 Level AA color contrast compliance. Text must have a minimum contrast ratio of 4.5:1 for normal text and 3:1 for large text.

6. Implement loading states and skeleton screens while data is being fetched. Users should understand that content is loading.

Performance Requirements

1. The blog must efficiently load and display the initial blog list. Initial page load should be completed in under 3 seconds on a standard internet connection.

2. Implement pagination or infinite scroll for the blog posts list to avoid loading all posts at once. This prevents performance degradation as the dataset grows.

3. Use the TrackBy function in all *ngFor loops to optimize change detection and improve rendering performance.

4. Implement change detection strategy OnPush in components where possible to reduce unnecessary change detection cycles.

5. Consider implementing lazy loading for images in the photo archive to improve initial load time.

6. Minimize bundle size by avoiding unnecessary imports and ensuring proper tree-shaking of unused code.

Angular Best Practices & Code Quality

1. Use strongly typed forms with the Reactive Forms or signal forms API. Avoid template-driven forms. No “any” types allowed.

2. Implement proper service architecture with dependency injection. Services should have clear, single responsibilities.

3. Use RxJS observables and reactive patterns instead of promises. Properly handle subscriptions (use async pipe or unsubscribe in ngOnDestroy).

4. Implement proper error handling with error interceptors, user-friendly error messages, and fallback states.

5. Use lazy loading for feature modules. The application should have a properly structured module architecture.

6. No console errors or warnings in the browser console when the application runs normally.

7. Provide unit tests with at least 70% code coverage. Tests should validate actual behavior, not just check if code executes.

Testing & Code Quality

1. All code must follow consistent naming conventions and be well-commented
2. Use ESLint and follow Angular style guide
3. Avoid code duplication and use DRY principles
4. Components should be reusable and properly documented